NLP + ML + LLM Systems

Project #03

News Category Classification

Text Classification • Multi-Class (42 Classes) • Google Colab Edition
Kaggle Datasets • Cloud LLM

Dataset

Name	News Category Dataset
Source	https://www.kaggle.com/datasets/rmisra/news-category-dataset
Details	200,000+ news articles with category labels (Politics, Entertainment, Tech, etc.)
Input	Headline + short description
Target	News category — multi-class, 42 classes

System Requirements

1. Data Pipeline

Load dataset from Kaggle using the Kaggle API.

Preprocessing Steps

- ✓ Lowercase text
- ✓ Remove punctuation, digits, special characters
- ✓ Tokenize using NLTK
- ✓ Remove stopwords
- ✓ Lemmatization via WordNet
- ✓ Handle class imbalance — SMOTE or class_weight

2. Feature Engineering

- ✓ TF-IDF (unigram + bigram)

Additional Features

- ✓ Word count
- ✓ Character count
- ✓ Punctuation count
- ✓ ....

Training Procedures

- ✓ Train/test split
- ✓ Cross-validation
- ✓ Hyperparameter tuning — GridSearchCV or RandomizedSearch

Metrics

- ✓ Accuracy
- ✓ Precision
- ✓ Recall
- ✓ F1-score
- ✓ ROC-AUC
- ✓ Confusion Matrix
- ✓ Model comparison table

Visualization & Analysis

1. Data Understanding (Before Modeling)

- ✓ Dataset shape (rows, columns)
- ✓ Data types (numerical, categorical, text, etc.)
- ✓ Missing values analysis
- ✓ Duplicate samples check
- ✓ Basic statistics (mean, median, std, min, max)

2. Target / Label Analysis

- ✓ Class distribution (bar chart)
- ✓ Check imbalance (important for classification)

3. Feature Analysis

Numerical Features

- ✓ Distribution plots (Histogram)
- ✓ Boxplots (outlier detection)
- ✓ Correlation matrix heatmap

Categorical Features

- ✓ Value counts (bar chart)
- ✓ Category vs target relationship (grouped bar chart)

Text Data (NLP)

- ✓ Token length distribution
- ✓ Most frequent words (top N)
- ✓ Word cloud (optional)
- ✓ N-grams analysis

4. Data Quality Checks

- ✓ Outliers visualization
- ✓ Skewness detection
- ✓ Feature scaling need (before/after visualization)
- ✓ Noise / anomalies inspection

Pre-processing Pipeline

Processing Order

Lowercase → Remove noise (punctuation / URLs / HTML) → Tokenise → Stop-word removal → Lemmatise

Representation

For Classical ML Models

- ✓ TF-IDF features (unigram + bigram)

Advanced Representation — Pretrained Embeddings (Bonus)

- ✓ Contextual embeddings using Hugging Face Transformers
- ✓ Use pretrained models (e.g., BERT, RoBERTa)
- ✓ Leverage embedding layers to generate dense semantic representations

Embedding Pipeline

Sentence → BERT → 768-dim vector → SVM classifier

ML Models (minimum 5)

Classical Models

- ✓ Logistic Regression
- ✓ Support Vector Machine (SVM)
- ✓ K-Nearest Neighbors (KNN)
- ✓ Decision Tree

Ensemble Methods

- ✓ Random Forest (bagging-based ensemble of decision trees)
- ✓ AdaBoost (adaptive boosting algorithm)

LLM Integration (Bonus)

Use ONE Cloud LLM: OpenAI GPT, Groq, or Claude.

Use LLM For

- ✓ Explain why an article belongs to a specific news category

Prompt Template

Example Prompt

Explain why the following news article is classified into the given category using simple reasoning:

```
Article: {text}
Category: {label}
Confidence: {score}
```

Frontend — Gradio on Colab

- ✓ Use Gradio with a public Colab share link
- ✓ Text input field for the news headline and description
- ✓ Display predicted category, confidence, explanation, and similar articles
- ✓ Runs entirely inside Google Colab — no external server needed

Project Structure (Example)

Folder Layout

```
news-category-ai/
├── data/
├── notebooks/
├── src/
│   ├── preprocessing.py
│   ├── features.py
│   ├── train_ml.py
│   ├── evaluate.py
│   ├── llm.py
├── models/
├── app_gradio.py
├── requirements.txt
└── README.md
```

Output Requirements

- ✓ Clean, modular Python code with clear comments throughout
- ✓ requirements.txt file listing all dependencies
- ✓ README.md with full setup and usage instructions

✓ Gradio demo application (Colab-ready for easy execution)

✓ Presentation slides explaining the idea (no code included)

✓ Written explanation document of the concept in simple words

✓ Structured Jupyter Notebook outlining the project workflow (code-focused, minimal explanations)